

Probability and Random Variables

- Out there in reality is a **generating process** that is running
- When we observe the outcomes of that process, we **sample** from it and create **data**
- When we **estimate/fit** models to the data, we **learn** about the process
- How do we expect our models to behave?
- How much variability is there in our predictions?
- These concepts are inherently probabilistic, so there's a payoff to explicitly using probability to model and analyze

Dice

- Let's roll two six-sided dice
- There are 36 possible outcomes:

(1, 1)	(1, 2)	(1, 3)	(1, 4)	(1, 5)	(1, 6)
(2, 1)	(2, 2)	(2, 3)	(2, 4)	(2, 5)	(2, 6)
(3, 1)	(3, 2)	(3, 3)	(3, 4)	(3, 5)	(3, 6)
(4, 1)	(4, 2)	(4, 3)	(4, 4)	(4, 5)	(4, 6)
(5, 1)	(5, 2)	(5, 3)	(5, 4)	(5, 5)	(5, 6)
(6, 1)	(6, 2)	(6, 3)	(6, 4)	(6, 5)	(6, 6)

- Which outcomes correspond to a roll whose sum is even? Strictly less than 7? The two dice match?
- What proportion of the time do those events occur?

Sets

- A set is a collection of things; we leave it at that
- Consider two sets E and F
 - The **union** $E \cup F$ are the elements in either E or F
 - The **intersection** $E \cap F$ are the elements in both E and F
 - The **complement** of E in F is $F \setminus E = E^c$, the set of elements in F but not E
- The **empty set** is denoted $\emptyset$, like a zero with a line through it, and has no elements

Probability Spaces

- There is a set of **outcomes** or **sample space** or **state space**, $\mathcal{S}$
- A set of **events**, $\mathcal{E}$, made of subsets of $\mathcal{S}$
- A probability function, p mapping $\mathcal{E}$ into $[0, 1]$

The combination, $(\mathcal{S}, \mathcal{E}, p)$, is a **probability space**

Events

- Events are combinations of outcomes, like "Roll an even number" or "The roll sums to seven"
- The set of events $\mathcal{E}$ has rules:
 - Nothing happening, $\emptyset$, is an event with probability 0
 - Something happening, $\mathcal{S}$, is an event with probability 1
 - For any event E , its complement $\mathcal{S} \setminus E = E^c$ is in $\mathcal{E}$
 - If E and F are sets in $\mathcal{E}$, then so are the union $E \cup F$ and the intersection $E \cap F$

Additivity

- **Union becomes sum, as long as you don't double count events**
- A fundamental law of probability is:

$$p(E \cup F) = p(E) + p(F) - \underbrace{p(E \cap F)}_{\text{Correction for "double counting"}}$$

- Notice, if $E \cap F = \emptyset$, and $p(\emptyset) = 0$, then

$$p(E \cup F) = p(E) + p(F)$$
- Notice, $p(E \cup F) \leq p(E) + p(F)$, so p is generally **sub-additive**

Finite Additivity

- **The probability of a collection of disjoint events is the sum of the probabilities of the events**
- Iterating on additivity gives us finite additivity:
We can break events up into disjoint sub-events, and sum
- If $E_1 \cap F = \emptyset$, then $p(\emptyset) = 0$, and
 $p(E_1 \cup F) = P(E_1) + P(F)$
- If $F = E_2 \cup E_3$ and $E_2 \cap E_3 = \emptyset$, the above implies

$$p(E_2 \cup E_3) = p(E_2) + p(E_3),$$

$$p(E_1 \cup E_2 \cup E_3) = p(E_1) + p(E_2) + p(E_3),$$

and so on, so that if $E_1, \dots, E_n$ are all mutually disjoint,

$$p\left(\bigcup_{i=1}^n E_i\right) = \sum_{i=1}^n p(E_i)$$

- If there is potential double counting of outcomes/events, how would you correct the above formula?

Complementary Events

- **The probability of an event "not happening" is 1 minus the probability it happens**
- Because E and its complement E^c are disjoint by definition and $\mathcal{S} = E \cup E^c$,

$$\begin{aligned} p(\mathcal{S}) &= p(E \cup E^c) \\ &= p(E) + p(E^c) \\ 1 &= p(E) + p(E^c) \\ 1 - p(E) &= p(E^c) \end{aligned}$$

- So the probability of the complementary event, is 1 minus the probability of the event

Conditional Probability

- **Information is about renormalizing to account for a shrinkage of the possibilities**
- The conditional probability of A given B is

$$p[A|B] = \frac{p[A \cap B]}{p[B]}$$

- We sum the probability that A and B occur together, and then renormalize by $p[B]$ to ensure our probabilities account for the knowledge that B has actually happened

Independence

- Event A is **independent** of event B if $p[A \cap B] = p[A]p[B]$, so the probability of their joint occurrence is not determined by whether or not the other event occurred

Conditional Expectation

- If we know an event B has occurred, we can renormalize the probabilities of different outcomes, and compute the expected value:

$$\mathbb{E}[X|B] = \sum_x x \times p[X = x|B] = \sum_x x \times \frac{p[X = x \text{ and } B]}{p[B]}$$

Probability Trees

When we string together probability spaces, we get **probability trees**. A **probability tree** is

1. A set of nodes that correspond to probability spaces
2. A set of edges that correspond to the outcomes in the probability space at each node

This makes a new probability space: - The set of all paths through the graph is the set of outcomes
- The set of all possible subsets of complete paths through the trees are the events - The probability of any path through the graph is the product of the probabilities along the edges

This turns a sequence of uncertainties into a "super-model" that nests all the uncertainties

Example: Two Period Binomial Asset Pricing Model

This is the core model of quantitative finance:

1. The initial stock price is S_0
2. With probability p , the stock price goes up by a factor of $(1 + u)$; with probability $1 - p$, the price goes down by a factor of $(1 - d)$
3. Repeat step 2

Are the price changes independent, or not? What is the probability that the final price exceeds S_0 ?

Example: Base Rate Fallacy

1. The patient is sick with probability i , and healthy with probability $1 - i$
2. The test is correct with probability p , and incorrect with probability $1 - p$

Are the patient's health status and the correctness of the test independent? Conditional on getting a positive test, what is the probability the patient is sick?

In []:

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

ps = [.8, .9, .95, .99]
is_ = np.linspace(0, .1, 20)

df = pd.DataFrame([{"i": i, "p": p,
                    "posterior": (i * p) / (i * p + (1 - i) * (1 - p))}
                  for p in ps
                  for i in is_
                  ])

sns.lineplot(
    data=df,
    x="i",
    y="posterior",
```

```
        hue="p",  
    )  
  
plt.xlabel("Base rate:  $i = pr[sick]$ ")  
plt.ylabel("pr[sick|+]" )  
plt.ylim(0, 1)  
plt.show()
```